

Diagramma delle classi e scelte progettuali

1. Descrizione delle classi

Sono state individuate le classi principali del progetto: **Torneo**, **Squadra**, **Giocatore**, **Partita**, **Classifica**, **RigaClassifica** e **ArchivioTorneo**.

La classe **Torneo** contiene l'elenco delle squadre iscritte e l'elenco delle partite registrate. Attraverso questa classe è possibile aggiungere, modificare ed eliminare squadre e partite e ottenere la classifica aggiornata.

La classe **Squadra** rappresenta una squadra partecipante al torneo. Ogni squadra possiede un nome e una lista di giocatori. Poiché i requisiti impongono un massimo di 5, ogni squadra è associata a 5 giocatori.

La classe **Giocatore** rappresenta un singolo giocatore appartenente a una squadra. Ogni giocatore è descritto da nome, cognome e data di nascita.

La classe **Partita** rappresenta una partita disputata tra due squadre. Questa contiene la data, la squadra di casa, la squadra ospite e il risultato finale scritto tramite i goal segnati da entrambe le squadre.

La classe **Classifica** ha il compito di calcolare e gestire la classifica del torneo. Essa contiene una lista di oggetti **RigaClassifica**, ciascuno dei quali rappresenta le statistiche di una singola squadra.

La classe **RigaClassifica** contiene i dati necessari per rappresentare la posizione di una squadra in classifica, come punti, partite giocate, vittorie, pareggi, sconfitte, goal fatti e goal subiti.

La classe **ArchivioTorneo** si occupa della persistenza dei dati. Questa permette di salvare e caricare le informazioni del torneo tramite un file locale, utilizzato come archivio dell'applicazione.

2. Relazioni tra le classi

Nel diagramma sono presenti relazioni di composizione e associazione.

La classe **Torneo** è collegata tramite composizione alle classi **Squadra** e **Partita**, perché il torneo contiene direttamente l'elenco delle squadre e delle partite. Un torneo può avere zero o più squadre e zero o più partite.

La classe **Squadra** è collegata tramite composizione alla classe **Giocatore**, poiché ogni squadra è formata da cinque giocatori.

La classe **Partita** è associata alla classe **Squadra** due volte: una volta per rappresentare la squadra di casa e una volta per rappresentare la squadra ospite. Ogni partita, coinvolge esattamente due squadre.

La classe **Classifica** è collegata tramite composizione alla classe **RigaClassifica**, perché la classifica è formata da più righe, ognuna riferita a una squadra.

La classe **RigaClassifica** è associata alla classe **Squadra**, dato che ogni riga della classifica riguarda una specifica squadra.

La classe **ArchivioTorneo** ha una dipendenza dalla classe **Torneo**, perché utilizza l'oggetto **Torneo** per effettuare le operazioni di salvataggio e caricamento.

3. Scelte progettuali

Per il progetto gli elementi principali del torneo, come squadre, giocatori, partite e classifica, sono stati modellati come oggetti separati. Una scelta importante riguarda la classe **Torneo**, che funge da classe principale per la gestione delle operazioni del sistema. In questo modo le funzionalità principali, come l'aggiunta di squadre e partite, sono concentrate in un unico punto. La classifica non viene gestita come dato statico, ma viene calcolata a partire dalle partite registrate. Questa scelta permette di evitare incoerenze tra risultati delle partite e posizione delle squadre in classifica. È stata inoltre introdotta la classe **RigaClassifica** per separare la gestione della classifica generale dai dati specifici di ogni squadra. In questo modo la classe **Classifica** si occupa del calcolo complessivo, mentre ogni riga conserva le statistiche della singola squadra. La persistenza è stata separata dalla logica principale tramite la classe **ArchivioTorneo**. Questa scelta consente di mantenere distinta la gestione dei dati su file dalla gestione del torneo, rendendo il sistema più ordinato e più semplice da modificare (come richiesto dai requisiti).

4. Coesione e accoppiamento

Le classi sono state progettate cercando di mantenere un'elevata coesione così che ogni classe ha una responsabilità precisa (ad esempio, **Squadra** gestisce i dati relativi alla squadra, **Partita** rappresenta una singola partita, mentre **ArchivioTorneo** si occupa solamente del salvataggio e del caricamento dei dati). L'accoppiamento tra le classi invece è stato mantenuto il più basso possibile separando le responsabilità principali. La logica del torneo infatti è separata dalla persistenza e potrà essere separata anche dall'interfaccia grafica. In questo modo eventuali modifiche alla GUI o al sistema di salvataggio non richiedono cambiamenti importanti alle classi del modello. Questa organizzazione rende il progetto più leggibile, più semplice da testare e più facile da estendere in futuro.